Gradient Inversion Attacks in Federated Learning

Deep Leakage from Gradients: Reconstructing Client Data from Shared Updates

Contents

1 Introduction **2**
1.1 Federated Learning 2
1.2 Threat Model: Gradient Inversion 2
1.3 Two Lines of Attack: Optimization versus Closed-Form Analysis 3

Chapter 1

Introduction

1.1 Federated Learning

Federated Learning (FL) is a distributed training setup where many clients (e.g. phones, hospitals, organizations) collaboratively train one shared model without pooling their data on a central server — each client trains locally on its own private data and only sends back the resulting model update [1]. Formally, FL trains a shared model across K clients, where each client k keeps a private dataset $\mathcal{D}_k$ and the server optimizes the global objective

$$\min_{w} \; F(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w), \qquad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell(w; x_i, y_i), \tag{1.1}$$

where w are the shared parameters and $n_k = |\mathcal{D}_k|$, $n = \sum_k n_k$. In round t, the server sends w_t to a subset of clients; each client trains locally and returns an update $\Delta w_k^{(t)}$ (or, in the single-step case this project focuses on, a gradient $g_k^{(t)} = \nabla_w F_k(w_t)$ scaled by the learning rate). The server aggregates with the FedAvg rule

$$w_{t+1} = w_t + \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} \Delta w_k^{(t)}. \tag{1.2}$$

Only $\Delta w_k^{(t)}$ (equivalently $g_k^{(t)}$) ever leaves the client — never x_i or y_i directly. This is the basis of FL's informal privacy claim, and the claim this project tests.

1.2 Threat Model: Gradient Inversion

We consider an *honest-but-curious* attacker that follows the FL protocol but tries to extract private data from what it legitimately observes — either the server itself, or an eavesdropper on the client-server channel. In both cases the attacker sees a gradient

$$g = \nabla_w \, \ell\big(w_t;\, x, y\big), \tag{1.3}$$

computed on a private sample (x, y) through a known architecture. Since g is a *deterministic function* of (x, y) and the known weights w_t, it is not necessarily one-way: if it can be inverted, exactly or by search, (x, y) can be recovered from the gradient alone. This is the *gradient inversion* attack, first shown as "Deep Leakage from Gradients" [2]. It demonstrates that exchanging gradients instead of raw data is not by itself a privacy guarantee — only a cost that depends on architecture, batch size, and whatever defense sits on top of the raw gradient.

1.3 Two Lines of Attack: Optimization versus Closed-Form Analysis

Attacks that invert Equation (1.3) fall into two families. This project implements and compares one method from each.

Optimization-based. Starting from a random dummy sample (x', y'), these attacks minimize the distance between its gradient and the leaked one,

$$\min_{x',y'} \; D\big(g', g\big), \qquad g' = \nabla_w \, \ell(w_t; x', y'), \tag{1.4}$$

by gradient descent. DLG [2] uses an L_2 distance and optimizes x', y' jointly; iDLG [3] recovers y' in closed form instead, leaving only x' to search. Inverting Gradients [4] uses a cosine-similarity cost, a total-variation prior, and signed Adam, scaling the attack up to ResNet-18. This project implements DLG/iDLG on LeNet (Part 1) and Inverting Gradients on ResNet-18 (Part 2).

Closed-form (analytical). R-GAP [5] skips search entirely: for a feed-forward network, the per-layer gradient equations let the input to each layer be solved recursively from the output layer back to the input, directly recovering x by linear algebra rather than gradient descent. This makes it far faster than optimization-based attacks when it applies, but more brittle — sensitive to architecture (bias-free layers, invertible activations) and, as shown later, to floating-point precision, since each layer is a hard constraint rather than a soft loss term. This project implements R-GAP on a LeNet-style CNN and a fully-connected network (Part 3), and evaluates both attack families against a DP-SGD-style defense [6] that clips and perturbs the gradient before it reaches the attacker.

Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS 2017), PMLR 54, pp. 1273–1282.

[2] L. Zhu, Z. Liu, S. Han. *Deep Leakage from Gradients*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019), pp. 14747–14756. arXiv:1906.08935.

[3] B. Zhao, K. R. Mopuri, H. Bilen. *iDLG: Improved Deep Leakage from Gradients*. arXiv:2001.02610, 2020.

[4] J. Geiping, H. Bauermeister, H. Dröge, M. Moeller. *Inverting Gradients — How Easy Is It to Break Privacy in Federated Learning?* Advances in Neural Information Processing Systems 33 (NeurIPS 2020).

[5] J. Zhu, M. B. Blaschko. *R-GAP: Recursive Gradient Attack on Privacy*. International Conference on Learning Representations (ICLR 2021). arXiv:2010.07733.

[6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang. *Deep Learning with Differential Privacy*. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS 2016), pp. 308–318. arXiv:1607.00133.